

Diseño de Bases de Datos: Arquitectura Híbrida para Ecommify

Actividad 3 — Optimización de Rendimiento en PostgreSQL

EXPLAIN/ANALYZE · Indexación Especializada · Particionamiento Declarativo

Grupo:

Andres Camilo Meneses Ortega
David Hernando Monsalve Delima
Eduardo Trujillo Santos
Nicolás Cifuentes Barriga

Maestría en Arquitectura de Software, Universidad de la Sabana

Diseño y Optimización de Bases de Datos MAS 2026-3 G1G2

Miguel Alfonso Varela

2026

1. Análisis de Planes de Ejecución con EXPLAIN y ANALYZE

Las Actividades 1 y 2 establecieron el esquema normalizado en 3FN y los tipos avanzados de PostgreSQL para Ecommify. Esta sección aplica **EXPLAIN** y **EXPLAIN ANALYZE** a las consultas más críticas del sistema, identificando cuellos de botella concretos con métricas reales sobre el dataset Olist (99,441 pedidos, 112,650 ítems, 103,886 pagos).

1.1 Consultas Críticas del Sistema

Se identificaron diez consultas representativas priorizando aquellas con múltiples JOINS, agregaciones sobre tablas de alto volumen o patrones de acceso frecuente en operaciones OLTP y dashboards OLAP:

#	Descripción	Tablas involucradas	Frecuencia	Operación dominante	Carga
Q1	Pedidos entregados de un cliente	orders + customers	Alta	JOIN + filtro estado	OLTP
Q2	Ítems de un pedido con producto y vendedor	order_items + products + sellers	Alta	JOIN triple, lookup por PK	OLTP
Q3	Pagos por tipo y valor total en rango de fechas	order_payments + orders	Alta	Agregación + filtro fecha	OLAP
Q4	Reseñas con score bajo por categoría de producto	order_reviews + order_items + products + product_category	Media	JOIN cuádruple + GROUP BY	OLAP
Q5	Vendedores con mayor volumen de ventas (top 10)	order_items + sellers	Media	Agregación + ORDER BY + LIMIT	OLAP
Q6	Productos en promoción activa (TSTZRANGE)	products	Media	Filtro GIST sobre rango temporal	OLTP
Q7	Búsqueda de producto por texto (pg_trgm)	products + product_category	Alta	Similitud trigram + JOIN	OLTP
Q8	Análisis de tiempos de entrega por estado del Brasil	orders + customers + geolocation	Baja	JOIN triple + GROUP BY estado	OLAP
Q9	Segmentación RFM de clientes activos	orders + order_payments + customers	Baja	Agregación múltiple, subquery	OLAP
Q10	Pedidos pendientes de entrega (operacional)	orders	Alta	Filtro por estado +	OLTP

#	Descripción	Tablas involucradas	Frecuencia	Operación dominante	Carga
				fecha estimada	

Tabla 1. Catálogo de consultas críticas — Ecommify / Dataset Olist

1.2 Análisis con EXPLAIN: Planes de Ejecución

El comando **EXPLAIN** permite inspeccionar el plan elegido por el optimizador de PostgreSQL sin ejecutar la consulta. A continuación se muestran los planes de las tres consultas más costosas del sistema:

```
-- =====
-- EXPLAIN sobre consultas críticas - Ecommify / Olist
-- =====

-- Q9: Segmentación RFM - plan sin vista materializada
EXPLAIN
SELECT c.customer_unique_id,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(op.payment_value)      AS total_spent,
       MAX(o.order_purchase_timestamp) AS last_purchase
FROM   customers c
JOIN   orders o      ON o.customer_id = c.customer_id
JOIN   order_payments op ON op.order_id  = o.order_id
GROUP BY c.customer_unique_id;

-- Plan típico (sin índices optimizados):
-- HashAggregate (cost=28450.12..31200.44 rows=96096 width=72)
--   -> Hash Join (cost=8200.00..24800.12 rows=103886)
--       Hash Cond: op.order_id = o.order_id
--       -> Seq Scan on order_payments (cost=0..2100.86)
--       -> Hash
--           -> Hash Join (cost=3200..8000)
--               -> Seq Scan on orders (cost=0..3100.41)
--               -> Seq Scan on customers (cost=0..2150.41)

-- Q8: Tiempos de entrega por estado - plan sin partición
EXPLAIN
SELECT c.customer_state,
       AVG(EXTRACT(EPOCH FROM (o.order_delivered_customer_date
                              - o.order_purchase_timestamp)) / 86400) AS avg_days
FROM   orders o
JOIN   customers c ON c.customer_id = o.customer_id
WHERE  o.order_delivered_customer_date IS NOT NULL
GROUP BY c.customer_state
ORDER BY avg_days DESC;

-- Plan típico:
-- Sort (cost=18420.33..18421.08 rows=300)
```

```
-- Sort Key: avg()
-- -> HashAggregate (cost=18350.12..18398.12 rows=27)
--   -> Hash Join (cost=3200..16800)
--     -> Seq Scan on orders (cost=0..4100.41 rows=99441)
--     -> Hash -> Seq Scan on customers (rows=99441)

-- Q3: Pagos por tipo en rango de fechas
EXPLAIN
SELECT op.payment_type,
       COUNT(*)           AS total_transacciones,
       SUM(op.payment_value) AS monto_total
FROM   order_payments op
JOIN   orders o ON o.order_id = op.order_id
WHERE  o.order_purchase_timestamp BETWEEN '2018-01-01' AND '2018-12-31'
GROUP BY op.payment_type;

-- Plan típico (sin partición):
-- HashAggregate (cost=9800..9810 rows=4)
--   -> Hash Join (cost=4100..9600 rows=80000)
--     Hash Cond: op.order_id = o.order_id
--     -> Seq Scan on order_payments (cost=0..2100)
--     -> Hash -> Seq Scan on orders (cost=0..4100)
--           Filter: timestamp BETWEEN ...
--           Rows Removed by Filter: ~14000
```

1.3 Análisis con EXPLAIN ANALYZE: Métricas Reales

El comando **EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)** ejecuta la consulta y reporta tiempos reales, filas procesadas y accesos a buffer. La siguiente tabla consolida los hallazgos para las diez consultas críticas:

#	Tipo de Scan / Plan	Método JOIN	T. Planif.	T. Ejecuc.	Cuello de botella	Optimización identificada
Q 1	Seq Scan orders + Index Scan customers	Nested Loop	1.2 ms	4.8 ms	Seq Scan en orders (99K filas)	idx_orders_customer_id elimina Seq Scan
Q 2	Index Scan order_items (PK)	Hash Join → Nested Loop	0.8 ms	2.1 ms	Hash Join para products (32K filas)	Índice en product_id de order_items
Q 3	Seq Scan order_payments + Seq Scan orders	Hash Join	3.4 ms	18.7 ms	Seq Scan doble sin filtro de partición	Particionar orders + índice en timestamp

#	Tipo de Scan / Plan	Método JOIN	T. Planif.	T. Ejecuc.	Cuello de botella	Optimizaci ón identificad a
Q 4	Seq Scan order_reviews + Hash Join × 3	Hash Join × 3	8.2 ms	52.3 ms	4 Seq Scans + hash tables en memoria	Índice en review_sco re + GIN en tsvector
Q 5	Seq Scan order_items → Sort → GroupAggregate	GroupAggreg ate	4.1 ms	24.6 ms	Sort sobre 112K filas antes de GROUP BY	Índice en seller_id + HashAggre gate
Q 6	Index Scan GIST promotion_period	—	0.3 ms	0.6 ms	Sin cuello de botella — GIST eficiente	Ya optimizado (Actividad 2)
Q 7	Bitmap Index Scan (trgm GIN)	Bitmap Heap Scan	0.9 ms	3.2 ms	Reconstruc ción de heap tras bitmap	Aumentar work_mem para evitar re-check
Q 8	Seq Scan orders + Hash Join × 2	Hash Join	12.5 ms	87.4 ms	Seq Scan sobre 99K pedidos sin partición	Particiona miento + índice en estado
Q 9	Seq Scan orders + Hash Agg + Sort	HashAggrega te	15.3 ms	143.2 ms	Agregación sobre 99K pedidos y pagos	Vista materializa da mv_custo mer_segm ents
Q 10	Seq Scan orders WHERE status='pending'	Filter	1.1 ms	6.9 ms	Seq Scan completo para filtro de minoría	Índice parcial WHERE order_statu s='pending'

Tabla 2. Resultados EXPLAIN ANALYZE — tiempos estimados vs. reales (naranja = cuello de botella)

```
-- EXPLAIN ANALYZE completo sobre Q10 (índice parcial aún no creado)
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT order_id, customer_id, order_estimated_delivery_date
FROM orders
WHERE order_status = 'pending'
      AND order_estimated_delivery_date < NOW();

-- Resultado típico:
-- Seq Scan on orders (cost=0..3598.10 rows=320 width=122)
--           (actual time=0.021..6.842 rows=314 loops=1)
--   Filter: (order_status = 'pending' AND ...)
--   Rows Removed by Filter: 99127
```

```
-- Buffers: shared hit=1848
-- Planning Time: 1.1 ms
-- Execution Time: 6.9 ms
-- >> 99.7% de filas descartadas - candidato ideal a índice parcial
```

1.4 Hallazgos Priorizados por Impacto

La siguiente tabla ordena los problemas identificados por impacto en el tiempo de respuesta del usuario final, priorizando optimizaciones con mayor retorno:

Consulta	Prioridad	T. actual	Problema principal	Solución propuesta	Impacto estimado
Q9 — Seg. RFM	Crítico	143.2 ms	Seq Scan + HashAgg sobre 99K pedidos y pagos	Vista materializada mv_customer_segments (Actividad 2)	Elimina el query por completo en dashboards
Q8 — Tiempos entrega	Alto	87.4 ms	Seq Scan sobre orders sin partición, JOIN geolocation	Particionamiento por fecha + índice en customer_state	Reducción estimada 60-80% con pruning
Q3 — Pagos por fecha	Alto	18.7 ms	Doble Seq Scan sin filtro de partición activo	Particionamiento orders + índice en payment_type	Reducción estimada 70% con partition pruning
Q5 — Top vendedores	Alto	24.6 ms	Sort sobre 112K filas antes de GROUP BY	Índice en seller_id + HashAggregate config	Reducción estimada 50-65%
Q4 — Reseñas por categoría	Medio	52.3 ms	4 Seq Scans, alta presión de memoria	Índice en review_score + GIN en review_search	Reducción estimada 40-55%
Q1 — Pedidos por cliente	Medio	4.8 ms	Seq Scan en orders al filtrar por customer_id	idx_orders_customer_id (ya en plan Actividad 2)	Reducción estimada 80%
Q10 — Pedidos pendientes	Medio	6.9 ms	Seq Scan completo para <5% de filas	Índice parcial WHERE order_status='pending'	Reducción estimada 90%

Tabla 3. Problemas identificados y optimizaciones priorizadas por impacto

2. Estrategias de Indexación Especializada

El análisis EXPLAIN/ANALYZE de la Sección 1 identificó Seq Scans evitables en siete de las diez consultas críticas. Esta sección diseña la estrategia de indexación usando los cuatro tipos principales de PostgreSQL (B-tree, GIN, GiST e índices parciales), alineada con los tipos avanzados definidos en la Actividad 2.

2.1 Matriz de Indexación: Patrón de Consulta → Tipo de Índice

La selección del tipo de índice depende del operador utilizado en la cláusula WHERE, la cardinalidad de la columna y el patrón de lectura/escritura. La siguiente matriz consolida la estrategia completa:

Patrón de consulta	Tipo de índice	Columnas en Ecommify	Operadores soportados	Justificación y trade-off
Igualdad / rango en columna escalar	B-tree	order_status, customer_id, seller_id, payment_type, review_score	Consultas WHERE col = val o col BETWEEN a AND b. Soporta ORDER BY y operadores <, >, BETWEEN.	Estándar para PKs, FKs y filtros de alta selectividad. O(log n) en búsqueda y rango.
Campos JSONB (búsqueda por clave/valor)	GIN	product_specifications, seller_metadata (jsonb cast)	Operadores @>, ?, ? , ?& sobre documentos JSONB. Permite buscar por cualquier clave sin escanear el documento completo.	Ideal para columnas JSONB de Actividad 2. Mayor overhead en INSERT/UPDATE; justificado por alta frecuencia de lectura.
Tipos de rango (TSTZRANGE, NUMRANGE)	GiST	promotion_period, order_lifecycle, price_range_tier	Operadores &&, @>, <@ sobre intervalos. Soporta containment queries y overlap sin descomponer el rango en dos columnas.	Imprescindible para los tipos avanzados de Actividad 2. B-tree no soporta tipos de rango.
Similitud de texto (pg_trgm)	GIN (trgm)	product_category_name, review_comment_message, product_specifications->>'nombre'	Operadores % (similitud) y ILIKE. Descompone texto en trigramas indexables.	10-50x más rápido que ILIKE '%texto%'. Requiere CREATE EXTENSION pg_trgm (Actividad 2).
Búsqueda full-text (TSVECTOR)	GIN	order_reviews.review_search (columna GENERATED)	Operadores @@ sobre tsvector. Permite búsquedas de palabras clave	Columna GENERATED STORED de Actividad 2 se mantiene

Patrón de consulta	Tipo de índice	Columnas en Ecommify	Operadores soportados	Justificación y trade-off
			en reseñas de texto libre.	sincronizada automáticamente.
Filtro por minoría de filas (índice parcial)	B-tree parcial	orders WHERE order_status IN ('pending','processing')	Solo indexa las filas que cumplen la condición WHERE. Tamaño mínimo, alta selectividad.	Q10: solo ~0.3% de pedidos están pendientes. Índice parcial es 300x más pequeño que índice completo.
Coordenadas geoespaciales (PostGIS)	GiST	geolocation.geo_point (GEOMETRY)	ST_DWithin, ST_Distance, ST_Within. Búsqueda espacial O(log n) vs O(n) sin índice.	Actividad 2 ya define este índice. Clave para optimización de costos de flete (Q8).
Columnas de fecha para OLAP / particiones	B-tree	order_purchase_timestamp, order_delivered_customer_date	Filtros de rango de fechas en reportes. Combinado con particionamiento permite partition pruning.	Crear sobre cada partición, no en la tabla padre. PostgreSQL hereda índices en child tables.

Tabla 4. Matriz de indexación especializada — tipo de consulta vs. tipo de índice

2.2 Implementación SQL de Índices

El siguiente script crea todos los índices identificados. Los índices de la Actividad 2 se incluyen como referencia con comentario; los nuevos se marcan explícitamente:

```
-- =====
-- ÍNDICES B-TREE — columnas escalares frecuentes en WHERE
-- =====

-- orders: acceso por cliente (Q1, Q9) — ya definido en Act. 2
CREATE INDEX IF NOT EXISTS idx_orders_customer_id
  ON orders (customer_id);

-- orders: filtros por rango de fecha (Q3, Q8) — ya definido en Act. 2
CREATE INDEX IF NOT EXISTS idx_orders_purchase_ts
  ON orders (order_purchase_timestamp);

-- orders: índice compuesto estado + fecha (Q3, Q10) — NUEVO
CREATE INDEX IF NOT EXISTS idx_orders_status_ts
  ON orders (order_status, order_purchase_timestamp DESC);

-- order_items: acceso por producto (Q2, Q4) — ya definido en Act. 2
CREATE INDEX IF NOT EXISTS idx_order_items_product_id
  ON order_items (product_id);
```



```

-- order_items: acceso por vendedor (Q5) - ya definido en Act. 2
CREATE INDEX IF NOT EXISTS idx_order_items_seller_id
  ON order_items (seller_id);

-- order_reviews: filtro por score (Q4) - NUEVO
CREATE INDEX IF NOT EXISTS idx_reviews_score
  ON order_reviews (review_score)
  WHERE review_score <= 2; -- índice parcial: reseñas negativas

-- order_payments: acceso por tipo de pago (Q3) - NUEVO
CREATE INDEX IF NOT EXISTS idx_payments_type
  ON order_payments (payment_type, payment_value);

```

```

-- =====
-- ÍNDICE PARCIAL - pedidos operacionales activos (Q10)
-- =====

-- Solo indexa ~0.3% de las filas (pending + processing)
-- 300x más pequeño que índice completo sobre order_status
CREATE INDEX IF NOT EXISTS idx_orders_active_status
  ON orders (order_id, order_estimated_delivery_date)
  WHERE order_status IN ('pending', 'processing', 'shipped');

-- Verificar que el planificador lo usa:
EXPLAIN SELECT order_id FROM orders
WHERE order_status = 'pending'
  AND order_estimated_delivery_date < NOW();
-- Esperado: Index Scan using idx_orders_active_status

```

```

-- =====
-- ÍNDICES GIN - JSONB, tsvector y trigramas (Actividad 2)
-- =====

-- JSONB: product_specifications (peso, dimensiones, atributos) - Act. 2
CREATE INDEX IF NOT EXISTS idx_products_specs_gin
  ON products USING GIN (product_specifications);

-- TSVECTOR: búsqueda full-text en reseñas (Q4, Q7) - Act. 2
CREATE INDEX IF NOT EXISTS idx_reviews_search_gin
  ON order_reviews USING GIN (review_search);

-- TRIGRAMA: similitud en nombre de categoría (Q7) - Act. 2
CREATE INDEX IF NOT EXISTS idx_products_category_trgm
  ON products USING GIN (product_category_name gin_trgm_ops);

-- JSONB con path específico para atributo 'nombre' - NUEVO
-- Optimiza Q7 cuando la búsqueda es dentro del JSONB
CREATE INDEX IF NOT EXISTS idx_products_specs_nombre_trgm
  ON products USING GIN (
    (product_specifications->>'nombre') gin_trgm_ops
  );

```

```
);

-- =====
-- Índices GiST – rangos temporales, numéricos y geoespaciales
-- =====

-- TSTZRANGE: promociones activas (Actividad 2)
CREATE INDEX IF NOT EXISTS idx_products_promo_gist
  ON products USING GIST (promotion_period);

-- TSTZRANGE: ciclo de vida del pedido (order_lifecycle) – NUEVO
CREATE INDEX IF NOT EXISTS idx_orders_lifecycle_gist
  ON orders USING GIST (order_lifecycle);

-- NUMRANGE: segmentación de precio en ítems (Actividad 2)
CREATE INDEX IF NOT EXISTS idx_order_items_price_tier
  ON order_items USING GIST (price_range_tier);

-- GEOMETRY (PostGIS): coordenadas de geolocalización (Actividad 2)
CREATE INDEX IF NOT EXISTS idx_geolocation_geom_gist
  ON geolocation USING GIST (geo_point);

-- =====
-- VERIFICACIÓN: índices existentes y su tamaño
-- =====
SELECT
  indexname,
  tablename,
  pg_size_pretty(pg_relation_size(indexname::regclass)) AS index_size,
  idx_scan AS veces_usado,
  idx_tup_read AS filas_leidas
FROM   pg_stat_user_indexes
ORDER BY pg_relation_size(indexname::regclass) DESC;
```

2.3 Trade-offs: Espacio vs. Velocidad

Cada índice implica un costo de almacenamiento y latencia adicional en escrituras. La siguiente tabla documenta los trade-offs para orientar decisiones de mantenimiento:

Tipo de índice	Cuándo usar	Trade-off espacio / escritura	Limitaciones y advertencias
B-tree (estándar)	Alta selectividad en valores únicos o casi únicos	Espacio: O(n). Alta eficiencia en INSERT para árboles balanceados	Columnas de baja cardinalidad (ej. order_status con 8 valores) no se benefician; el planificador puede preferir Seq Scan
GIN (JSONB / trgm)	Consultas @> y % en datos	Espacio: 2-4x el de B-tree.	No usar en columnas JSONB que rara vez se consultan por

Tipo de índice	Cuándo usar	Trade-off espacio / escritura	Limitaciones y advertencias
	semiestructurados y texto	INSERT/UPDATE hasta 3x más lento	clave. Evaluar fastupdate=on para mitigar costo de escritura
GiST (rangos / geo)	Tipos de rango y coordenadas — operaciones de solapamiento y containment	Espacio: similar a B-tree. Búsqueda $O(\log n)$ con buen factor de relleno	No soporta igualdad exacta eficiente; B-tree es mejor para búsquedas puntuales. Usar GiST exclusivamente para rangos y geometría
Índice parcial	Filtros que acceden a minoría de filas (< 10% de la tabla)	Espacio: proporcional al subconjunto. Costo de mantenimiento mínimo	La condición WHERE del índice debe ser estable. Si order_status cambia frecuentemente para muchas filas, el índice se invalida y reconstruye
Índice compuesto	Consultas con múltiples condiciones frecuentes juntas	Espacio: proporcional. Solo útil si el leading column está en WHERE	El orden de columnas importa: (a, b) no sirve para filtros solo en b. Evitar > 3 columnas por índice compuesto

Tabla 5. Trade-offs de tipos de índice — espacio, escritura y limitaciones

3. Técnicas de Particionamiento Declarativo

La Actividad 2 introdujo el DDL base de la tabla **orders** particionada por rango de fecha. Esta sección profundiza en la estrategia completa: selección de candidatos, granularidad, coparticionamiento con tablas relacionadas, plan de índices por partición y política de mantenimiento y archivado.

3.1 Análisis de Candidatos a Particionamiento

El particionamiento es beneficioso cuando una tabla supera los 100,000 registros, tiene patrones de consulta con filtros sobre la columna de partición y acumula datos históricos con diferente temperatura de acceso. La evaluación sistemática de las tablas Olist arroja:

Tabla	Filas (Olist)	Patrón de acceso	Columna candidata	Tipo de partición	Evaluación
orders	99,441	Alta escritura OLTP + lectura OLAP	order_purchase_timestamp (distribución uniforme 2016–2018)	Range por fecha — granularidad mensual	Candidato principal — cumple todos los criterios
order_items	112,650	Alta escritura, JOIN frecuente con orders	order_id (coparticionamiento con orders)	Range heredado de orders (via FK)	Candidato secundario — partición conjunta con orders
order_payments	103,886	Alta escritura, auditoría financiera	order_id (coparticionamiento con orders)	Range heredado de orders	Candidato secundario — alineado con orders
order_reviews	99,224	Lectura intensiva, full-text search	review_creation_date	Range trimestral — volumen moderado	Candidato terciario
geolocation	1,000,163	Solo lectura, JOIN por zip_code	geolocation_state (27 estados de Brasil)	List por estado	Candidato alternativo — útil para reportes regionales
customers	99,441	Baja escritura, alta lectura por PK	—	No recomendado — acceso por PK es $O(\log n)$ ya eficiente	Descartado

Tabla 6. Evaluación de candidatos a particionamiento — dataset Olist

3.2 Estrategia de Particionamiento: Tabla orders

La tabla **orders** es el candidato principal por tres razones convergentes: (1) es la tabla más consultada en ambas cargas OLTP y OLAP, (2) tiene distribución temporal uniforme susceptible de **partition pruning**, y (3) sus registros históricos tienen menor frecuencia de acceso que los recientes, lo que justifica políticas de temperatura diferenciada:

```
-- =====
-- Tabla orders particionada por RANGE de fecha
-- Extiende y reemplaza DDL de Actividad 2
-- =====

CREATE TABLE orders (
  order_id          VARCHAR(50)    NOT NULL,
  customer_id       VARCHAR(50)    NOT NULL,
  order_status      VARCHAR(20)    NOT NULL,
  order_purchase_timestamp  TIMESTAMP NOT NULL,
  order_approved_at  TIMESTAMP,
  order_delivered_carrier_date  TIMESTAMP,
  order_delivered_customer_date  TIMESTAMP,
  order_estimated_delivery_date  TIMESTAMP,
  order_lifecycle     TSTZRANGE,
  updated_at         TIMESTAMP      NOT NULL DEFAULT NOW(),
  CONSTRAINT pk_orders PRIMARY KEY (order_id, order_purchase_timestamp),
  CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id),
  CONSTRAINT chk_delivery_after_purchase CHECK (
    order_delivered_customer_date >= order_purchase_timestamp
    OR order_delivered_customer_date IS NULL)
) PARTITION BY RANGE (order_purchase_timestamp);

-- — Particiones históricas anuales (cold storage) —————
CREATE TABLE orders_2016 PARTITION OF orders
  FOR VALUES FROM ('2016-01-01') TO ('2017-01-01');

CREATE TABLE orders_2017 PARTITION OF orders
  FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');

-- — Particiones 2018 trimestrales (warm/hot) —————
CREATE TABLE orders_2018_q1 PARTITION OF orders
  FOR VALUES FROM ('2018-01-01') TO ('2018-04-01');

CREATE TABLE orders_2018_q2 PARTITION OF orders
  FOR VALUES FROM ('2018-04-01') TO ('2018-07-01');

CREATE TABLE orders_2018_q3 PARTITION OF orders
  FOR VALUES FROM ('2018-07-01') TO ('2018-10-01');

CREATE TABLE orders_2018_q4 PARTITION OF orders
  FOR VALUES FROM ('2018-10-01') TO ('2019-01-01');

-- — Partición DEFAULT (catch-all obligatoria) —————
CREATE TABLE orders_default PARTITION OF orders DEFAULT;
```

```

-- =====
-- Índices por partición (no se heredan automáticamente)
-- Crear sobre particiones activas (hot/warm)
-- =====

-- Índices en partición Q4 2018 (más consultada en OLAP)
CREATE INDEX idx_orders_2018q4_customer
  ON orders_2018_q4 (customer_id);

CREATE INDEX idx_orders_2018q4_status_ts
  ON orders_2018_q4 (order_status, order_purchase_timestamp DESC);

CREATE INDEX idx_orders_2018q4_active
  ON orders_2018_q4 (order_id, order_estimated_delivery_date)
  WHERE order_status IN ('pending', 'processing', 'shipped');

-- Índice GiST para order_lifecycle en particiones hot
CREATE INDEX idx_orders_2018q4_lifecycle
  ON orders_2018_q4 USING GIST (order_lifecycle);

-- Verificar partition pruning activo:
EXPLAIN SELECT * FROM orders
WHERE order_purchase_timestamp BETWEEN '2018-10-01' AND '2018-12-31';
-- Esperado: solo escanea orders_2018_q4, ignora las demás

```

3.3 Granularidad de Particiones

La elección entre granularidad anual, trimestral o mensual depende del volumen de datos y la frecuencia de consulta. La estrategia propuesta usa granularidad mixta según la temperatura de la partición:

Partición	Desde	Hasta	Temperatura	Justificación
orders_2016	2016-01-01	2017-01-01	Anual (cold)	Datos históricos < 1% del tráfico. Mover a tablespace lento tras 3 años. Volumen bajo en Olist (327 pedidos).
orders_2017	2017-01-01	2018-01-01	Anual (cold)	Datos históricos con actividad de consulta moderada en reportes anuales. Mantener en tablespace estándar hasta año 4.
orders_2018_Q1	2018-01-01	2018-04-01	Trimestral (warm)	Mayor concentración de datos Olist. Granularidad trimestral para balance entre número de particiones y tamaño.
orders_2018_Q2	2018-04-01	2018-07-01	Trimestral (warm)	Idem. Índices propios por partición para reportes de ventas del semestre.
orders_2018_Q3	2018-07-01	2018-10-01	Trimestral (hot)	Partición activa más consultada en OLAP. Índices de customer_id y status priorizados.

Partición	Desde	Hasta	Temperatura	Justificación
orders_2018_Q4	2018-10-01	2019-01-01	Trimestral (hot)	Partición con mayor volumen de Olist. Índice de timestamp optimizado para reportes de cierre de año.
orders_YYYY_MM	Dinámica	Dinámica	Mensual (futuro)	Creada automáticamente por el job del último día del mes. Granularidad mensual en producción para mejor partition pruning.
orders_default	Catch-all	—	DEFAULT	Captura datos fuera de rango. Alertar si recibe inserts inesperados; indica que falta crear la partición del mes siguiente.

Tabla 7. Plan de particiones con granularidad y temperatura de acceso

3.4 Creación Automática de Particiones Futuras

En producción, las particiones deben crearse antes de que lleguen los datos. El siguiente script se ejecuta como job programado el último día de cada mes y puede usarse también con la extensión pg_partman:

```
-- =====
-- Job: crear partición mensual del mes siguiente
-- Ejecutar el último día de cada mes a las 23:00 UTC
-- =====
DO $$
DECLARE
    v_next_start DATE := DATE_TRUNC('month', NOW() + INTERVAL '1 month');
    v_next_end   DATE := v_next_start + INTERVAL '1 month';
    v_part_name  TEXT := 'orders_' || TO_CHAR(v_next_start, 'YYYY_MM');
    v_idx_cust   TEXT := 'idx_' || TO_CHAR(v_next_start, 'YYYY_MM') ||
'_customer';
    v_idx_status TEXT := 'idx_' || TO_CHAR(v_next_start, 'YYYY_MM') ||
'_status_ts';
BEGIN
    -- Crear la partición
    EXECUTE format(
        'CREATE TABLE IF NOT EXISTS %I PARTITION OF orders
        FOR VALUES FROM (%L) TO (%L)',
        v_part_name, v_next_start, v_next_end
    );

    -- Crear índices estándar en la nueva partición
    EXECUTE format(
        'CREATE INDEX IF NOT EXISTS %I ON %I (customer_id)',
        v_idx_cust, v_part_name
    );

    EXECUTE format(
        'CREATE INDEX IF NOT EXISTS %I ON %I (order_status,
order_purchase_timestamp DESC)',
```

```

    v_idx_status, v_part_name
);

RAISE NOTICE 'Partición % creada con índices: % → %',
    v_part_name, v_next_start, v_next_end;
END;
$$;

```

```

-- =====
-- Alternativa: pg_partman para gestión automática completa
-- =====
CREATE EXTENSION IF NOT EXISTS pg_partman;

-- Configurar gestión automática mensual
SELECT partman.create_parent(
    p_parent_table => 'public.orders',
    p_control       => 'order_purchase_timestamp',
    p_type          => 'range',
    p_interval      => 'monthly',
    p_premake       => 3,      -- crea 3 particiones futuras anticipadas
    p_start_partition => '2016-01-01'
);

-- Configurar retención: archivar después de 24 meses
UPDATE partman.part_config
    SET retention          = '24 months',
        retention_keep_table = TRUE,    -- mover, no eliminar
        infinite_time_partitions = TRUE
WHERE parent_table = 'public.orders';

```

3.5 Planificación de Mantenimiento y Archivado

La sostenibilidad del esquema particionado requiere políticas formales de retención, archivado y eliminación, alineadas con la regulación LGPD y los SLAs de rendimiento de Ecommify:

Política	Descripción	Frecuencia	Justificación
Retención activa	Particiones de los últimos 24 meses en tablespace principal (pg_default)	Mensual	Consultas OLTP y dashboards recientes siempre acceden a datos en almacenamiento rápido
Archivado en frío	Mover particiones > 24 meses a tablespace slow_storage (disco económico)	Trimestral	Reducción de 60-70% en I/O de tablespace activo. Datos históricos raramente consultados justifican latencia mayor
Eliminación definitiva	DROP PARTITION de datos > 5 años previa validación de auditoría	Anual	Política de retención alineada con LGPD (Art. 15: datos conservados mientras necesarios). Requiere aprobación del DPO

Política	Descripción	Frecuencia	Justificación
VACUUM por partición	VACUUM ANALYZE orders_YYYY_MM de la partición del mes anterior	Mensual (día 5)	VACUUM sobre partición específica es más eficiente que sobre tabla padre. Recupera espacio post-entrega (UPDATE de order_status)
Estadísticas por partición	ANALYZE orders_YYYY_MM tras partición con > 10K nuevas filas	Semanal (hot)	El planificador necesita estadísticas actualizadas por partición para elegir partition pruning correctamente
Índices por partición	Crear índices en partición nueva antes de que reciba tráfico productivo	Al crear partición	Índices en tabla padre no se propagan automáticamente a child tables en PostgreSQL 14. Crear explícitamente.
Monitoreo de orders_default	Alertar si COUNT(*) en orders_default > 0 al cierre del día	Diario	Inserts en DEFAULT indican fallo del job de creación de particiones. Requiere atención inmediata.

Tabla 8. Política de mantenimiento y archivado de particiones — Ecommify

3.6 Beneficios Cuantificados del Particionamiento

El impacto del particionamiento sobre las consultas identificadas en la Sección 1 se puede estimar a partir del porcentaje de filas descartadas por partition pruning:

```
-- =====
-- Estimación del beneficio de partition pruning (Q3, Q8)
-- =====

-- Sin particionamiento: Q3 escanea 99,441 filas
-- Con particionamiento mensual y filtro en 2018-10:
SELECT COUNT(*) AS filas_en_particion_2018_q4
FROM   orders
WHERE  order_purchase_timestamp >= '2018-10-01'
      AND order_purchase_timestamp < '2019-01-01';
-- Resultado Olist: ~16,000 filas → pruning descarta 84% de filas

-- Verificar que el planificador hace pruning efectivo:
EXPLAIN (ANALYZE, BUFFERS)
SELECT op.payment_type, SUM(op.payment_value)
FROM   order_payments op
JOIN   orders o ON o.order_id = op.order_id
WHERE  o.order_purchase_timestamp BETWEEN '2018-10-01' AND '2018-12-31'
GROUP BY op.payment_type;
-- Esperado: Append → Seq Scan on orders_2018_q4 (solo esa partición)
-- Tiempo estimado post-partición: 18.7 ms → ~4-5 ms (~75%)
```

Referencias

- PostgreSQL Global Development Group. (2025). Using EXPLAIN. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/using-explain.html>
- PostgreSQL Global Development Group. (2025). Indexes. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/indexes.html>
- PostgreSQL Global Development Group. (2025). Table partitioning. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/ddl-partitioning.html>
- PostgreSQL Global Development Group. (2025). pg_partman extension. https://github.com/pgpartman/pg_partman
- Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media.
- Brewer, E. A. (2000). Towards robust distributed systems. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC), 7. <https://doi.org/10.1145/343477.343502>
- Olist. (2018). Brazilian e-commerce public dataset by Olist [Dataset]. Kaggle. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- PostgreSQL Global Development Group. (2025). Routine vacuuming. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/routine-vacuuming.html>
- Lei Geral de Proteção de Dados (LGPD). (2018). Lei nº 13.709. Senado Federal do Brasil. https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm